Payment Intelligence Ecosystem

Nền tảng dữ liệu thanh toán: độ ổn định giao dịch và rủi ro gian lận

Đỗ Trung Tuyến

12/11/2026

Báo cáo thực tập · demo 6 tuần + Stage G

Nội dung

1. Bài toán và kiến trúc

2. Product 01 · Payment Reliability

3. Product 02 · Fraud & Risk

4. Stage G · Từ điểm số đến quyết định (thăm dò)

5. Kỹ thuật, chất lượng và demo

6. Hạn chế và hướng tiếp theo

Bài toán và kiến trúc

Bài toán: hai câu hỏi của một hệ thống thanh toán

Product 01: Payment Reliability

Hệ thống có đang chạy ổn không?

Tỷ lệ thành công, độ trễ, lỗi theo nhà cung cấp (provider); **phát hiện sớm sự cố**.

Dữ liệu: **giả lập** theo contract `payment-event`.

Product 02: Fraud & Risk

Giao dịch này có rủi ro gian lận không?

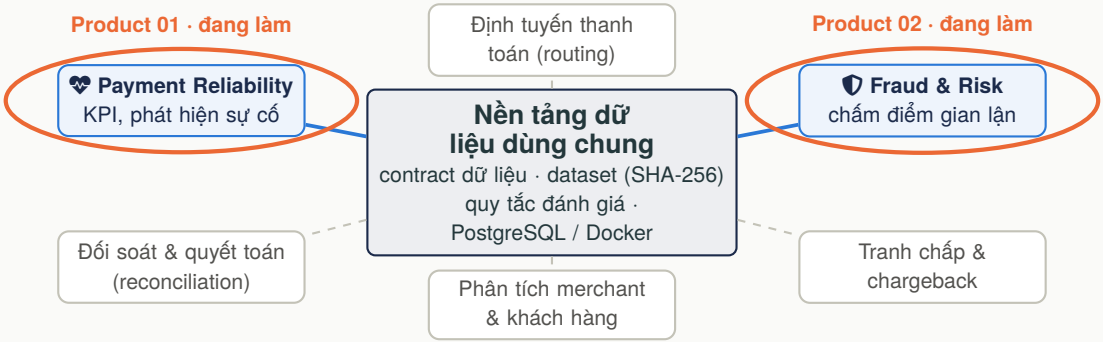
Chấm điểm rủi ro để người vận hành **ưu tiên kiểm tra**; không tự động chặn.

Dữ liệu: **IEEE-CIS** (Kaggle, công khai).

Tiêu chí thành công

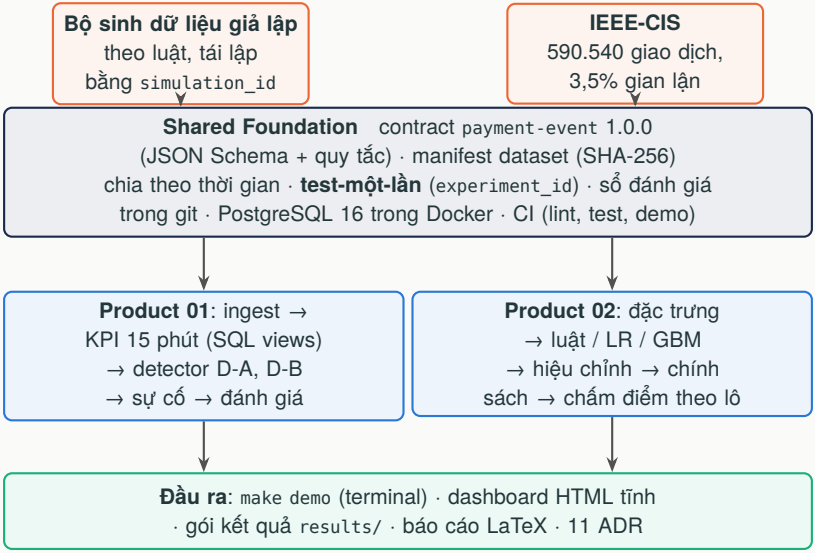
Đánh giá **trung thực**, **tái lập được**, **giải thích được từng bước**. Điểm số mô hình là ưu tiên thứ yếu. Đây là demo, không phải hệ thống production.

Hệ sinh thái thanh toán: đề tài chọn 2 trong 6 hệ thống con

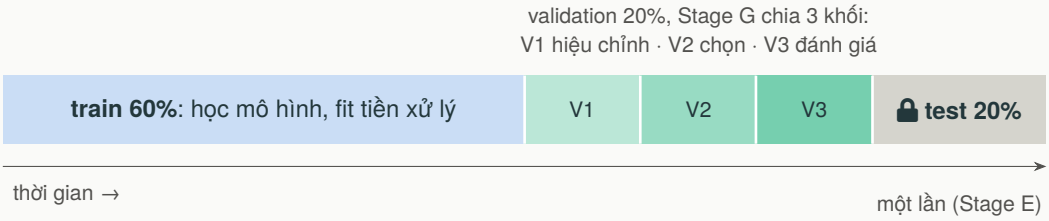


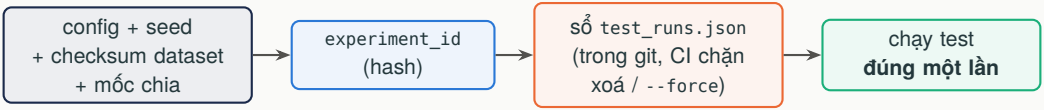
Khoanh cam: hai sản phẩm trong phạm vi. Ô xám, nét đứt: hướng mở rộng, **minh hoạ, chưa làm**. Hai sản phẩm **chung nền tảng, không chung dữ liệu** (ADR 0002): không có dataset công khai nào có cả dữ liệu vận hành lẫn nhãn gian lận với chung khoá giao dịch.

Kiến trúc: nguồn dữ liệu → nền tảng → hai pipeline → đầu ra



Nguyên tắc đánh giá: chia theo thời gian, tập test chỉ mở một lần

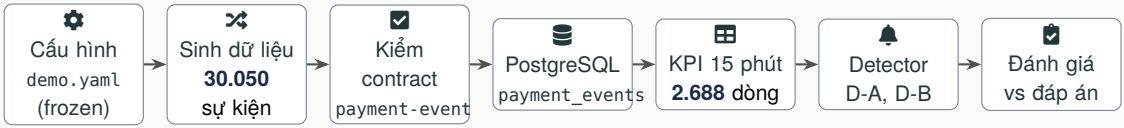




Phiên bản thư viện, commit, môi trường nằm ở `execution_id` riêng (ADR 0010): nâng thư viện không tạo hash mới nên không “mở lại” được tập test.

Product 01 · Payment Reliability

Pipeline Product 01



Dữ liệu (bộ tuning)

7 ngày · 3 provider
25.573 payment, **30.050** attempt
4 sự cố cài sẵn (đáp án)

`sim-485cacc9c858`

KPI (SQL views)

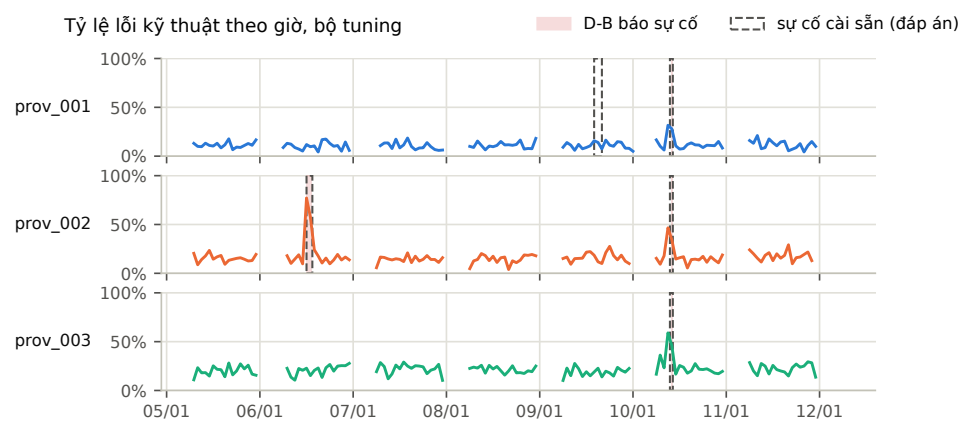
Tỷ lệ thành công **76,5%**
Thành công lần đầu **77,1%**
Lỗi kỹ thuật **16,9%**
Độ trễ trung vị theo khung

Chạy lại

```
make up
make demo
```

kết quả: terminal + dashboard; tự so với số công bố

Tín hiệu: tỷ lệ lỗi kỹ thuật theo giờ và sự cố



Mỗi provider một hàng, cùng thang. Vùng đỏ: D-B báo sự cố; khung nét đứt: sự cố cài sẵn (đáp án, người vận hành thật không có). Ẩn giờ có dưới 20 attempt.

Hai detector

D-A: z-score bền vững (đường cơ sở)

So mỗi khung 15 phút với trung vị và MAD của cùng khung giờ những ngày trước.

- đơn giản, dễ giải thích
- nhạy với khung ít attempt ⇒ báo nhầm

D-B: nhị thức gộp cửa sổ (chính)

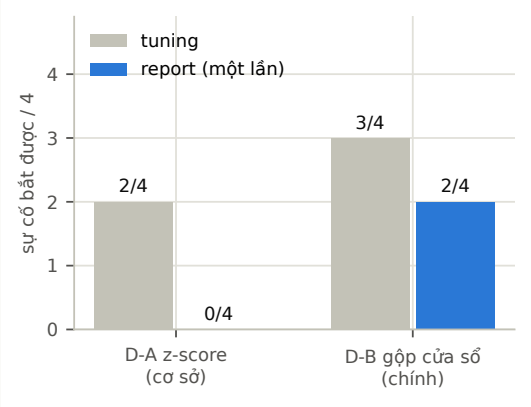
Gộp số lỗi / số attempt trong một cửa sổ, kiểm định nhị thức so với đường nền 24 giờ.

- tính đúng cỡ mẫu nhỏ
- ngưỡng chỉnh trên bộ tuning, rồi khoá

Quy trình chống “chỉnh theo đáp án”

Ngưỡng chỉnh trên bộ **tuning**; bộ **report** (seed khác, có một loại sự cố chưa gặp) đánh giá **một lần** bằng `make detect-report`, ghi sổ `detect_report_runs.json`.

Kết quả Product 01



	Tuning	Report (trích lại)
D-B (chính)	**3/4** · 0,0	**2/4** · 0,0
D-A (cơ sở)	2/4 · 1,0	0/4 · 2,0

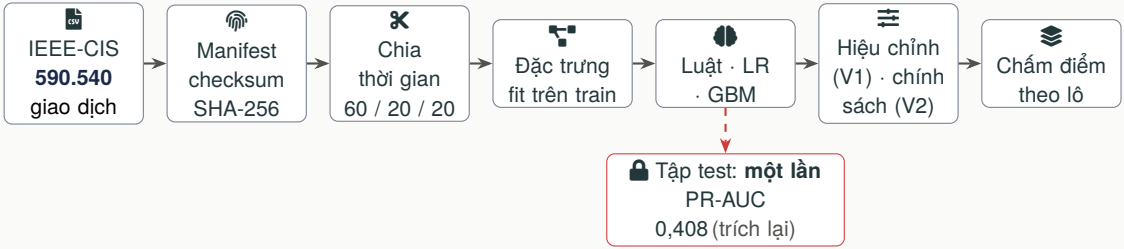
sự cố bắt được · báo nhầm/ngày

- D-B: không báo nhầm ở cả hai bộ; bắt trong 15–30 phút
- Sự cố x1,5 nằm dưới giới hạn phát hiện
- Report thấp hơn tuning: đúng như kỳ vọng khi khoá ngưỡng trước

`det-3ddc26cff253 · det-08335c6af086`

Product 02 · Fraud & Risk

Pipeline Product 02



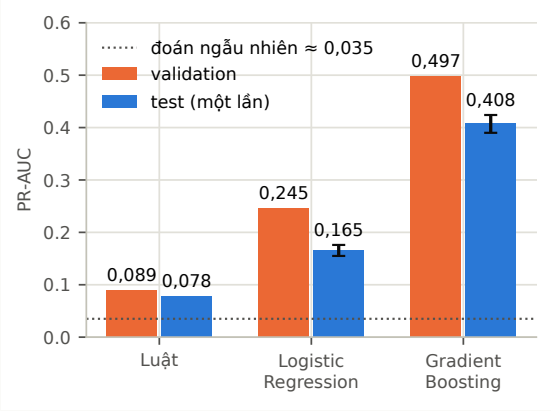
Chống rò rỉ

Nhãn tách khỏi đặc trưng; đặc trưng chỉ dùng quá khứ; mọi tiền xử lý fit trên train; ngưỡng chọn trên validation.

Không dùng accuracy

Gian lận chỉ ≈3,5%: báo "không gian lận" cho mọi giao dịch đã đúng 96,5%. Dùng **PR-AUC** (kèm CI bootstrap), recall tại FPR cố định.

Chất lượng xếp hạng: ba mô hình



Mô hình	Valid	Test (trích lại)	R@FPR5%
Luật	0,089	0,078	0,22
Logistic	0,245	0,165	0,28
GBM	**0,497**	**0,408**	**0,51**

GBM test 95% CI [0,390; 0,424]

- GBM gấp ≈12 lần mức đoán ngẫu nhiên
- Valid > test: validation đã bị dùng để chọn
- Recall@FPR5% chọn ngưỡng bằng nhãn của chính tập test: đo *thứ tự*, không phải hiệu quả triển khai

`run-d6faa4699008 · run-f8807cb6f780`

Ngưỡng đã chốt trước khi mở test, và thí nghiệm rò rỉ

GBM, ngưỡng 0,853 (chọn trên validation)

Precision (test)	**0,516**
Recall (test)	**0,371**
FPR (test)	**1,24%**

Điểm vận hành FPR 1,24% khác điểm FPR 5% ở slide trước: không đọc là recall “tụt từ 51% xuống 37%”.

Thí nghiệm rò rỉ (validation)

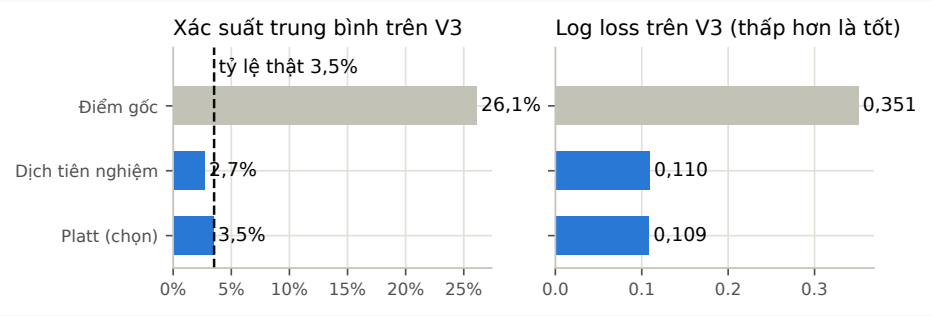
Mã hoá `card1` bằng nhãn: PR-AUC **+0,063**
Chia ngẫu nhiên thay vì theo thời gian: **−0,068**

Rò nhãn làm điểm *đẹp giả*. Chia ngẫu nhiên lại cho điểm *thấp hơn*: phần lớn chênh lệch do hai tập validation không cùng dòng.

`leak-827d929eaa73`

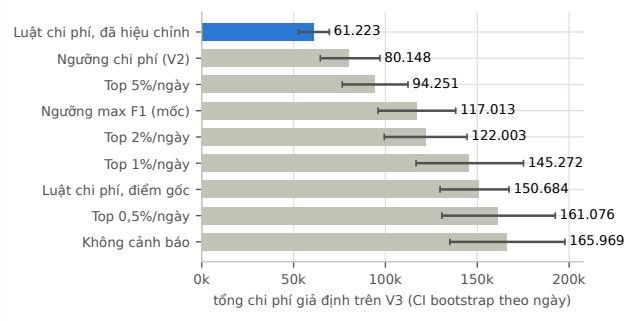
Stage G · Từ điểm số đến quyết định (thăm dò)

Điểm có đọc được như xác suất không? Hiệu chỉnh



- Điểm gốc trung bình **26,1%**, gian lận thật **3,5%**: **thổi lên ≈7 lần** (do `class_weight="balanced"`).
- Platt fit trên V1, chọn trên V2, đo trên V3: log loss **0,351 → 0,109**.
- Hiệu chỉnh đơn điệu **không đổi PR-AUC**; nó chỉ quan trọng khi đặt ngưỡng theo chi phí.

So sánh chính sách quyết định (chi phí giả định)



`pol-af5df43efe54 · V3 · L0.75/R5/F1`

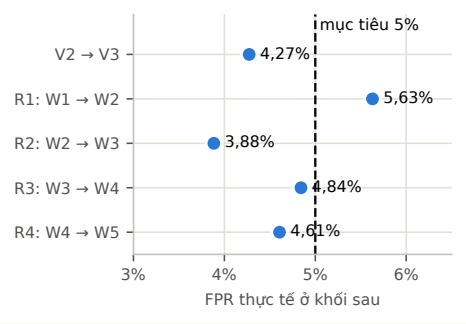
Chi phí giả định (đơn vị tương đối):

- bỏ sót: 0,75× số tiền
- mỗi lần duyệt: 5
- mỗi cảnh báo nhầm: 1

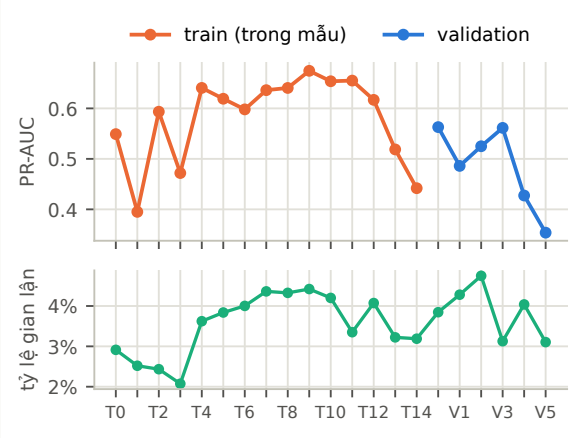
Luật chi phí dùng xác suất **đã hiệu chỉnh** rẻ nhất ở **27/27** tổ hợp chi phí.

Phân tích độ nhạy, không phải ROI. Một phần do luật được chấm bằng chính hàm chi phí nó tối ưu.

Ngưỡng chọn hôm nay có giữ được ngày mai? Độ ổn định

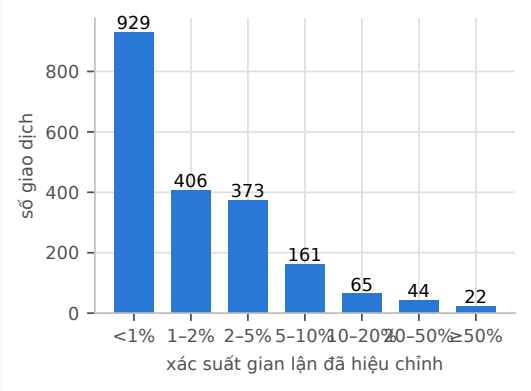


Ngưỡng FPR 5% chọn ở khối trước cho FPR thực tế **3,9–5,6%** ở khối sau.



PR-AUC tuần validation **0,35–0,56**. PSI 4 đầu vào < 0,05: trôi phân phối *không* giải thích được hai tuần cuối.

Chấm điểm theo lô (G08)



`make score-model` rồi `make score`

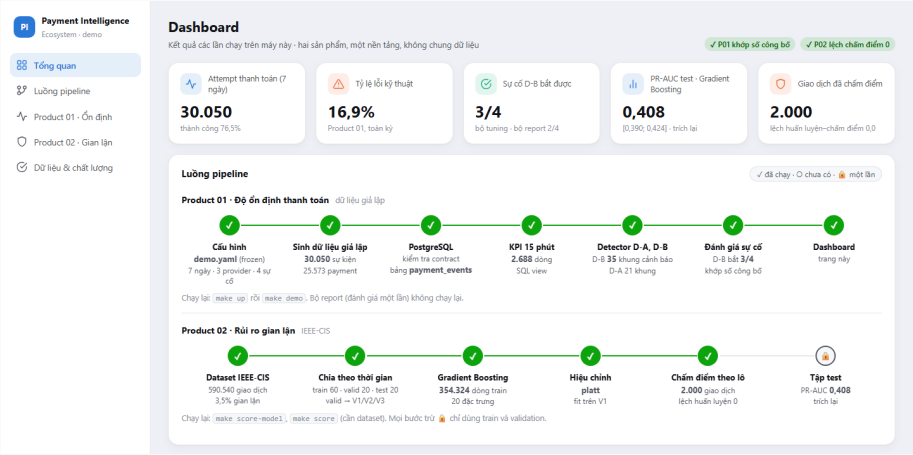
- **2.000** giao dịch V3 **không nhãn**
- lệch huấn luyện–chấm điểm: **0** (cùng dòng, cùng điểm)
- ghi `model_version`, `feature_version`, `execution_id`
- **từ chối** file có cột nhãn hoặc dòng thuộc giai đoạn test

Phá hoại có chủ đích: lệch `hour_of_day` 7 giờ lúc chấm ⇒ 2 test báo lỗi.

`score-19e94be9616d`

Kỹ thuật, chất lượng và demo

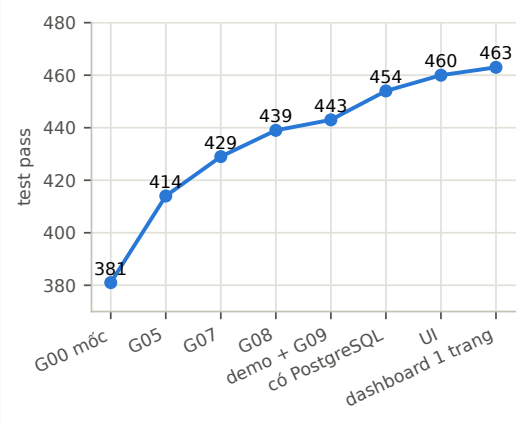
Dashboard: một trang, mở offline



`make demo` → `artifacts/dashboard/index.html`: số liệu chính, luồng pipeline, biểu đồ có tooltip, sáng/tối.

Số test chỉ trích từ `results/published.json`.

Chất lượng phần mềm



- **463** test pass, 1 skip (có PostgreSQL thật)
- CI: lint + test trên Python 3.11 và 3.13, PostgreSQL 16, job `make demo`, job kiểm sổ test-một-lần
- **11** ADR cho mọi quyết định kiến trúc
- Tự kiểm tra mỗi task: đảo nhãn, nhìn tương lai, tính lại độc lập, phá hoại có chủ đích, chạy hai lần trùng từng byte

Demo trong 5 phút

`make setup`	môi trường + git hook
`make up`	PostgreSQL 16 (Docker)
`make demo`	cả hai sản phẩm

Terminal (thư viện `rich`, ADR 0011): sản phẩm đang chạy, % tiến độ, bảng kết quả, danh sách kiểm tra ✔ / ✘.

Kiểm tra tự động khi demo

- ✔ D-B 3/4, 0 báo nhầm/ngày (khớp công bố)
- ✔ D-A 2/4, 1,0 báo nhầm/ngày
- ✔ lệch chấm điểm = 0
- ✔ không đụng bộ report, không đọc test

Product 02 cần dataset IEEE-CIS trong `datasets/raw/` (không commit).

Hạn chế và hướng tiếp theo

Hạn chế: kết quả này *không* chứng minh

- Product 01 dùng dữ liệu **giả lập**, mỗi bộ chỉ **4 sự cố**: kiểm chứng trên bất thường đã biết, không phải tỷ lệ phát hiện thực tế.
- Product 02: `C1`–`C14` mang gần hết tín hiệu (bỏ chúng: PR-AUC validation 0,497 → 0,158), ý nghĩa chưa xác minh.
- Validation đã dùng nhiều lần (lưới GBM, rò rỉ, Stage G): số trên validation có thể **lạc quan**.
- Chi phí Stage G là **giả định**: phân tích độ nhạy, không phải ROI.
- Giả định nhãn gian lận có ngay; thực tế chargeback đến muộn nhiều tuần.
- Hai sản phẩm chưa chung dữ liệu; không API, không giám sát mô hình, không huấn luyện lại.

Hướng tiếp theo

Ngắn hạn

- Merge chuỗi PR Stage G vào `main`, xác nhận CI xanh
- G10 (chờ duyệt): sự cố *tăng dần* để thử D-B có bị "luộc ếch"; nạp trùng / muộn / hỏng, tải 10×; lật nhãn x% trên train

Dài hạn (ngoài phạm vi demo)

- Nối hai sản phẩm qua `payment_id` từ cùng nguồn sự kiện
- Mở rộng sang hệ thống con khác (đối soát, chargeback)
- API chấm điểm, giám sát trôi mô hình

Cảm ơn đã lắng nghe!

Hỏi đáp

github.com/TrungTuyenDo02/Payment_Intelligence_Ecosystem